MongoDB

BASIC TASK DOCUMENTATION

Terminal screenshots arranged in assignment order

Tasks Covered

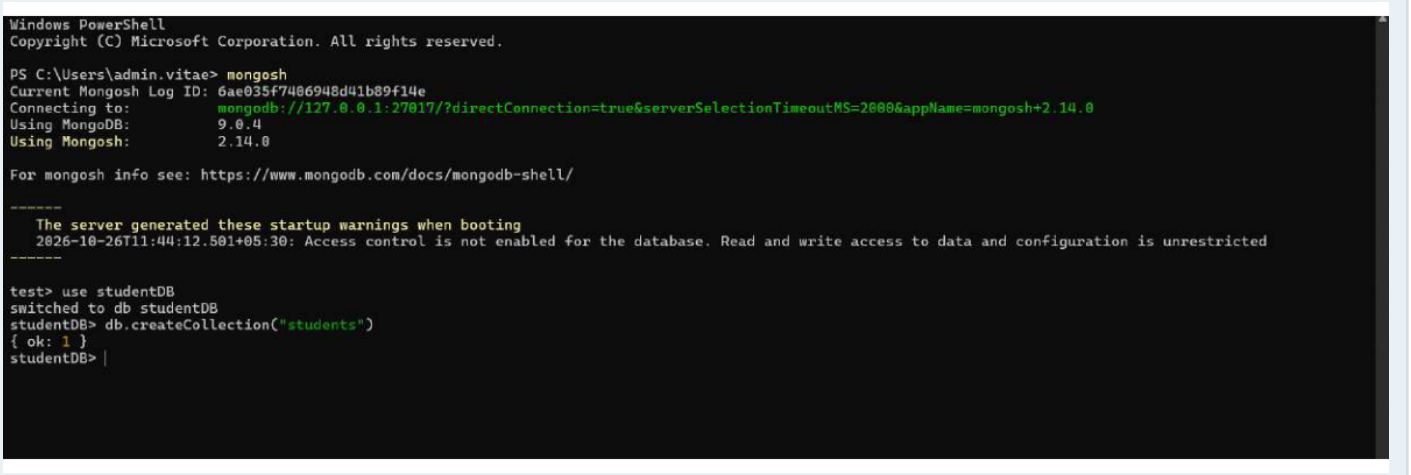
- 1. Database & Collection Setup
- 2. Insert Operations
- 3. Read Operations
- 4. Update Operations
- 5. Delete Operations
- 6. Query Operators
- 7. Library System Use Case

Screenshots are kept as provided; headings and layout have been added for clarity.

TASK 1

Database & Collection Setup

Create the studentDB database and students collection.



TASK 2

Insert Operations

Insert one document and multiple student documents.

```
studentDB> db.students.insertOne({name:"Arun", age:21, course:"MERN Stack", status:"ongoing"})
{
  acknowledged: true,
  insertedId: ObjectId('6ae03899406948d41b89f14f')
}
studentDB> db.students.insertMany([
studentDB> db.students.insertMany([
|   { name: "Priya", age: 22, course: "Python", status: "ongoing" },
|   { name: "Karthik", age: 19, course: "MERN Stack", status: "completed" },
|   { name: "Divya", age: 24, course: "Java", status: "ongoing" },
|   { name: "Rahul", age: 26, course: "MERN Stack", status: "ongoing" }])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ae03a01406948d41b89f150'),
    '1': ObjectId('6ae03a01406948d41b89f151'),
    '2': ObjectId('6ae03a01406948d41b89f152'),
    '3': ObjectId('6ae03a01406948d41b89f153')
  }
}
studentDB>
```

TASK 3

Read Operations — Fetch All Documents

Display all documents in the students collection using find().

```
studentDB> db.students.find()
[
  {
    _id: ObjectId('6ae038994069 48d41b89f14f'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f150'),
    name: 'Priya',
    age: 22,
    course: 'Python',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f151'),
    name: 'Karthik',
    age: 19,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f152'),
    name: 'Divya',
    age: 24,
    course: 'Java',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f153'),
    name: 'Rahul',
    age: 26,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
```

TASK 3

Read Operations — Filter Documents

Retrieve specific student records using a query filter.

```
]
studentDB> db.find({course:"MERN Stack"})
TypeError: db.find is not a function
studentDB> db.students.find({course:"MERN Stack"})
[
  {
    _id: ObjectId('6ae03899406948d41b89f14f'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'ongoing'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f151'),
    name: 'Karthik',
    age: 19,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f153'),
    name: 'Rahul',
    age: 26,
    course: 'MERN Stack',
    status: 'ongoing'
  }
]
studentDB>
```

TASK 4

Update Operations

Update one document and multiple documents, then verify the results.

```
studentDB> db.students.updateOne({name:"Priya"},{$set:{status:"completed"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB> db.students.updateMany({course:"MERN Stack"},{$set:{status:"completed"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 2,
  upsertedCount: 0
}
studentDB> db.students.find()
[
  {
    _id: ObjectId('6ae03899406948d41b89f14f'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f150'),
    name: 'Priya',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f151'),
    name: 'Karthik',
    age: 19,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
```

TASK 6

Query Operator — $gt

Find documents where a field value is greater than the specified value.

```
studentDB> db.students.find({age:{$gt:20}})
[
  {
    _id: ObjectId('6ae03899406948d41b89f14f'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f150'),
    name: 'Priya',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f153'),
    name: 'Rahul',
    age: 26,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03e26406948d41b89f154'),
    name: 'Divya',
    age: 24,
    course: 'Java',
    status: 'ongoing'
  }
]
```

TASK 6

Query Operator — $lt

Find documents where a field value is less than the specified value.

```
studentDB> db.students.find({age:{$lt:25}})
[
  {
    _id: ObjectId('6ae03899406948d41b89f14f'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f150'),
    name: 'Priya',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f151'),
    name: 'Karthik',
    age: 19,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03e26406948d41b89f154'),
    name: 'Divya',
    age: 24,
    course: 'Java',
    status: 'ongoing'
  }
]
```

TASK 6

Query Operator — $in

Match documents where a field value is included in a list.

```
studentDB> db.students.find({course:{$in: ["MERN Stack","Python"]}})
[
  {
    _id: ObjectId('6ae03899406948d41b89f14f'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f150'),
    name: 'Priya',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f151'),
    name: 'Karthik',
    age: 19,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f153'),
    name: 'Rahul',
    age: 26,
    course: 'MERN Stack',
    status: 'completed'
  }
]
```

TASK 6

Query Operators — Condition Filter

Run a query using a conditional operator.

```
studentDB> db.students.find({$and:[{age:{$gt:25}},{course:"MERN Stack"}]})
[
  {
    _id: ObjectId('6ae03a01406948d41b89f153'),
    name: 'Rahul',
    age: 26,
    course: 'MERN Stack',
    status: 'completed'
  }
]
```

TASK 6

Query Operators — Combined Filter

Run a query using multiple conditions.

```
studentDB> db.students.find({$or:[{status:"completed"},{age:{$lt:20}}]})
[
  {
    _id: ObjectId('6ae03899406948d41b89f14f'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f150'),
    name: 'Priya',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f151'),
    name: 'Karthik',
    age: 19,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f153'),
    name: 'Rahul',
    age: 26,
    course: 'MERN Stack',
    status: 'completed'
  }
]
```

TASK 6

Query Operator — $exists / Verification

Check field existence and inspect updated student records.

```
studentDB> db.students.updateOne({name:"Arun"},{$set:{email:"arun@gmail.com"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentDB> db.students.find({email:{$exists:true}})
[
  {
    _id: ObjectId('6ae03899406948d41b89f14f'),
    name: 'Arun',
    age: 21,
    course: 'MERN Stack',
    status: 'completed',
    email: 'arun@gmail.com'
  }
]
studentDB> db.students.find({email:{$exists:false}})
[
  {
    _id: ObjectId('6ae03a01406948d41b89f150'),
    name: 'Priya',
    age: 22,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f151'),
    name: 'Karthik',
    age: 19,
    course: 'MERN Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ae03a01406948d41b89f153'),
    name: 'Rahul',
    age: 26,
    course: 'MERN Stack',
    status: 'completed'
```

TASK 5

Delete Operations

Delete one document, delete documents for practice, and verify the collection.

```
studentDB> db.students.deleteOne({name:"Divya"})
{ acknowledged: true, deletedCount: 1 }
studentDB> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 4 }
studentDB> db.students.find()

studentDB>
```

TASK 7

Use Case — Library System Setup

Create the library database collection and insert book records.

```
test> use SchoolDB
switched to db SchoolDB
SchoolDB> db.createCollection("author")
{ ok: 1 }
SchoolDB> db.author.insertMany([[{title:"Wings of Fire", author: "APJ Abdul Kalam", genre: "Biography", price:350, copiesAvailable: 5, isIssued:false},{titlt
titltitle:"Ponniyin Selvan", author: "Kalki", genre: "Historical", price:600, copiesAvailable: 3, isIssued:false}, {title:"Atomic Habits", author:"James Cle
ar", genre: "Self-help", price:450, copiesAvailable:4, isIssued:false},{title:"The Alchemist", author: "Paulo Coelho", genre: "Fiction", price:300, copiesAv
copiesAvailacopiesAvailable:2,isIssued:true}]])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ae04d15964e4750ac8869d1'),
    '1': ObjectId('6ae04d15964e4750ac8869d2'),
    '2': ObjectId('6ae04d15964e4750ac8869d3'),
    '3': ObjectId('6ae04d15964e4750ac8869d4')
  }
}
```

TASK 7

Use Case — Update a Book Record

Update a library record and check the operation result.

```
SchoolDB> db.author.updateOne({ title: "Atomic Habits" },{ $set: { isIssued: true }, $inc: { copiesAvailable: -1 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

TASK 7

Use Case — Search by Genre

Find books that belong to the Fiction genre.

```
SchoolDB> db.author.find({genre:"Fiction"})
[
  {
    _id: ObjectId('6ae04d15964e4750ac8869d4'),
    title: 'The Alchemist',
    author: 'Paulo Coelho',
    genre: 'Fiction',
    price: 300,
    copiesAvailacopiesAvailable: 2,
    isIssued: true
  }
]
```

TASK 7

Use Case — Search by Price

Search library records using a price condition.

```
SchoolDB> db.author.find({price:{$lt:500}})
[
  {
    _id: ObjectId('6ae04d15964e4750ac8869d1'),
    title: 'Wings of Fire',
    author: 'APJ Abdul Kalam',
    genre: 'Biography',
    price: 350,
    copiesAvailable: 5,
    isIssued: false
  },
  {
    _id: ObjectId('6ae04d15964e4750ac8869d3'),
    title: 'Atomic Habits',
    author: 'James Clear',
    genre: 'Self-help',
    price: 450,
    copiesAvailable: 3,
    isIssued: true
  },
  {
    _id: ObjectId('6ae04d15964e4750ac8869d4'),
    title: 'The Alchemist',
    author: 'Paulo Coelho',
    genre: 'Fiction',
    price: 300,
    copiesAvailacopiesAvailable: 2,
    isIssued: true
  }
]
```

TASK 7

Use Case — Combined Search

Use $and with conditions to find matching book records.

```
SchoolDB> db.author.find({$and:[{isIssued:false},{copiesAvailable:{$gt:0}}]})
[
  {
    _id: ObjectId('6ae04d15964e4750ac8869d1'),
    title: 'Wings of Fire',
    author: 'APJ Abdul Kalam',
    genre: 'Biography',
    price: 350,
    copiesAvailable: 5,
    isIssued: false
  },
  {
    _id: ObjectId('6ae04d15964e4750ac8869d2'),
    titltitle: 'Ponniyin Selvan',
    author: 'Kalki',
    genre: 'Historical',
    price: 600,
    copiesAvailable: 3,
    isIssued: false
  }
]
SchoolDB>
```